

This work is licensed under a Creative Commons license



Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Software Reverse Engineering

Part II – Operating System Mechanisms and File Formats

Simone Aonzo, PhD



Main purposes of an Operating System (OS)

An OS is a software that manages computer's hardware and software resources, and provides common services for software apps.

Its **kernel** provides the core low-level functionalities.

- **Process/Thread Management** – multitasking, scheduling, synchronization
 - **Process**: the container for a running program, with its own memory space and resources
 - **Thread**: the unit of execution inside a process. Shares memory/resources but with its own stack and registers
- **Memory Management** – allocation, paging, virtual memory
- **File System Management** – organizing data storage
- **Device Management** – unified access to hardware devices
- **Security & Access Control** – permissions, isolation
- **System Calls Interface** – exposes controlled entry points for apps to request services

The CPU helps the OS

Each CPU mode has distinct rights for accessing memory, I/O and instructions.

In descending order of privileges:

- **x86 rings:**
 - Ring -2 – System Management Mode
 - Ring -1 – Hypervisor (Intel VT-x / AMD-V VMX)
 - Ring 0 – OS Kernel (**kernel** mode)
 - Rings 1 and 2 – exist in theory, but unused by modern OSes
 - Ring 3 – Applications (**user** mode)
- **ARM Exception Levels (EL):**
 - EL3 – Secure Monitor
 - EL2 – Hypervisor
 - EL1 – OS Kernel (**kernel** mode)
 - EL0 – Applications (**user** mode)

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

How software interacts with the OS

- System Call (**syscall**)
 - Direct request to the kernel using a *dedicated instruction*
 - x86-32: 'int' or 'sysenter', x86-64: 'syscall', ARM: 'svc'
 - And, a known value on a specific register
 - E.g., on Linux x86-64 'syscall' and 'rax=2' → 'sys_open()'
 - The CPU switches from user mode to kernel mode
 - User (x86 ring 3 / ARM EL0) ↔ Kernel (x86 ring 0 / ARM EL1)
- Library Call (**libcall**)
 - May be self-contained (e.g. `strlen()`)
 - Syscall wrapper
 - E.g., on Linux: `printf()` → `write()`
 - E.g., on Windows: `printf()` → `NtWriteFile()`

Key Difference

Syscall = actual kernel service, not used in traditional programming

Libcall = user-level function, sometimes a convenience wrapper

Windows APIs Suffixes and Prefixes

- Suffixes

- **Ex**: Extended → the function is an enhanced version of an older one
- **A**: ANSI → 8-bit character strings (`char*`)
- **W**: Wide character → 16-bit character strings (`wchar_t*`)
- E.g. `RegOpenKeyW` and `RegOpenKeyExA` of `advapi32.dll`

- Prefixes

- **Nt**: Native APIs of `ntdll.dll`
 - Syscall wrappers: User → Kernel
 - Not documented or supported for standard app development
- **Zw**: Used by kernel components (e.g. drivers)

Example: calling a Windows API

What happens when an app calls `CreateFileW()`?

- 1 The app's code calls `CreateFileW()`, exported by `kernel32.dll`
 - `kernel32.dll` is a thin wrapper that forwards the call to `KERNELBASE.dll`
- 2 `KERNELBASE.dll` performs parameter validation and prepares the native structures
- 3 `KERNELBASE.dll` then calls the native API `NtCreateFile()` in `ntdll.dll`
- 4 `ntdll.dll` executes the `syscall` instruction to transition to kernel mode
- 5 The kernel receives the request in `ntoskrnl.exe` and dispatches it to the appropriate kernel routine (e.g., I/O Manager)
- 6 The result propagates back: `ntdll.dll` → `KERNELBASE.dll` → `kernel32.dll` → app

Linux System Call Interface

- User-space Interfaces
 - **glibc**: GNU C Library, providing POSIX APIs
 - E.g., `open()` and `read()`
 - These are high-level wrappers for syscalls, like `Nt` functions in Windows
 - Stable and supposed to be used
 - Provide error handling, ABI compatibility, and portability
 - **Syscalls**: Exposed through `syscall()`
- Kernel-space Implementations
 - Each syscall has a corresponding kernel function
 - E.g., `sys_open()` and `sys_read()`

Strings

Windows uses UTF-16 internally, offering both ANSI (A) and Unicode (W) APIs, while Linux treats strings as byte sequences, typically using UTF-8 by convention without separate APIs or kernel-level encoding

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Processes vs Threads

- **Process**

- Container for a running program
- Own virtual address space, file descriptors/handles, security context
- Identified by a PID (Linux) or process object/ID (Windows)
- A process can start another process

- **Thread**

- Unit of execution *inside* a process
- Shares code, data, and open files/handles with other threads
- Has its own CPU registers, stack, thread ID, and scheduling state

- Each process is represented by a `task_struct` in the kernel
- Identified by:
 - **PID**: process ID
 - **PPID**: parent PID (process hierarchy)
- Creation:
 - `fork()` / `vfork()` → clone current process
 - `execve()` → replace code/data with a new program image
- Termination:
 - Normal exit via `exit()` / `_exit()`
 - Abnormal via signals (e.g., `SIGSEGV`, `SIGKILL`)
- `/proc/pid`: command line, environment, maps, FD list, etc.

“Threads are just processes that share more things”

- Threads of a process: `/proc/pid/task`
- Kernel API: `clone()` with flags:
 - `CLONE_VM`: share address space
 - `CLONE_FILES`: share file descriptors
 - `CLONE_SIGHAND`: share signal handlers, etc.
- POSIX threads (`pthread_*` APIs)
 - user-space wrapper over `clone()`
- Each thread has:
 - Its own stack and thread ID (TID)
 - Its own entry point function (start routine)

- Process = kernel **process object** + user-mode **PE image**
 - Own virtual address space and handle table
- Created with high-level APIs:
 - `CreateProcessW()`, `ShellExecuteEx()`, etc.
 - Under the hood: native APIs like `NtCreateUserProcess()`
- Important internal structures:
 - PEB (Process Environment Block)
 - image base, loader data, module list
 - Handle table: references to objects
 - files, events, sections, processes, ...

- Thread = kernel **thread object** scheduled on a CPU
 - Own stack(s) (user/kernel), registers, thread context
 - Belongs to exactly one process
- Creation APIs:
 - `CreateThread()`, `_beginthreadex()` (C runtime-friendly)
 - Native: `NtCreateThreadEx()` (often used by malware)
- User-mode structure: TEB (Thread Environment Block)
 - Thread-local storage (TLS), stack limits, SEH chain (x86), etc.

Linux vs Windows: Processes & Threads

- **Process Model** (broadly similar)
 - Lin: a task with its own resources (memory, FDs, namespaces)
 - Win: a process object with address space and handle table
- **Thread Model** (fundamentally different)
 - Lin: threads are “lightweight processes” (tasks) created via `clone()`
 - Win: threads are distinct kernel objects separate from processes
- **Creation & Execution**
 - Lin: `fork()`, `execve()`, and fine-grained `clone()`
 - Win: `CreateProcess` and `CreateThread` APIs
- **Scheduling**
 - Lin: scheduler treats each task the same (threads $\cong$ processes)
 - Win: threads are the scheduling unit with dynamic priority boosts
- **Introspection & Metadata**
 - Lin: exposed via `/proc` and kernel structures like `task_struct`
 - Win: exposed via PEB/TEB in user mode and `EPROCESS/ETHREAD` in the kernel
- **Exception, Loading & Hierarchy**
 - Lin: signals, ELF loader, strict parent/child tree
 - Win: SEH/APC, PE loader, no strong parent/child hierarchy

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Everything is a File

In Unix-like systems, resources are represented as **files**

- **Regular files** → data on disk
- **Directories** → special files mapping names to inodes
- **Devices** → appear in /dev (e.g. /dev/null)
- **Processes** → exposed in /proc/<pid>
- **Kernel settings** → via /proc and /sys

Key Idea

All resources can be accessed with the same system calls:
`open()`, `read()`, `write()`, `close()`

Traditional access control in Unix-like systems

Discretionary Access Control (DAC)

Security model where the owner (creator) of a resource decides (i.e., discretionary) who is allowed to access it, and with what permissions

- Users identified by **UID** (and groups by **GIDs**), non-negative integers
- Every process runs as some “user”
 - Not necessarily a human: many services have a corresponding user
 - **UID=0** AKA **root/superuser** is privileged...
- Files have
 - Owner
 - Group
 - **Access Control List (ACL)**
 - bit-mask representing **rxw** permissions for owner/group/others
- *however*, privileged processes (UID=0) bypass permission checks

The need for elevating privileges

For example:

- How can users change their passwords? `passwd(1)`
- Password hashes are stored in `/etc/shadow`
 - Only root can r/w
- How can `passwd` work? Or `sudo`?
- Executing “`set-user/group-ID programs`” replaces the `UID/GID` of the process. . .

... actually,

- Each process has actually three UIDs: **effective**, **real**, **saved**
 - inherited on `fork(2)`
- executing “set-user-ID programs” replaces the effective and saved UIDs with the owner's UID
- `set[e]uid(2)/set[e]gid(2)` allow processes to change their UIDs/GIDs

Dropping/re-gaining privileges is error-prone [TDSW08] and implementing the principle of least privilege can be difficult

Linux's enhancements

- **POSIX ACLs**, ACL(5), which are more fine-grained
 - Still, less flexible than Windows ACLs
- **Seccomp BPF** (SECure COMputing with filters)
 - Reduce the set of available system calls to processes
- **Linux Security Module** (LSM) framework
 - Lets the kernel plug in different security models
 - Each module can extend or restrict the default DAC, e.g.
 - Yama: prevents code injection between processes of the same user
 - SELinux: **Mandatory Access Control** implementation
 - Capabilities: finer granularity superuser-privileges system

Most LSM modules can be stacked. A nice overview is given in:

www.starlab.io/blog/a-brief-tour-of-linux-security-modules

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - **Windows**
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Everything is an Object

In Windows NT (“New Technology” since 1993), nearly all system resources are represented as **objects**, managed by the **Object Manager**

- **Objects**: files, processes, threads, registry keys, events, mutexes, etc.
- **Handles**: user-mode references to objects, used by applications
- **Access Control**: each object has a security descriptor (with ACLs)
- **System Calls**: programs request operations on objects via handles

Key Idea

Instead of “everything is a file” in UNIX-like, Windows adopts “**everything is an object**”, a unified abstraction with security and management built-in

Security Identifier (SID)

Security principals (e.g., users, groups, computers, ...) that can be referred to grant/deny permissions are **identified by Security IDs**

- SIDs are **variable-length values**, probabilistically **unique in the world**
- S-1-5-21-<machine-or-domain-id>-RID
 - S-1-5-21: base for most user account
 - <machine-or-domain-id>: unique ID for the system|domain
 - RID: relative Identifier → unique per account on that system|domain
 - e.g., S-1-5-21-2043073706-359421727-405887128-1000
- Except for some *well-known* SIDs:
 - S-1-0-0 is *Nobody*
 - S-1-1-0 is the *Everyone* group
 - S-1-5-32-544 is the *Administrators* group
- And some *well-known* **RIDs**:
 - 500: Administrator $\approx$ Unix root → S-1-5-21-<machine-or-domain-id>-500
 - 501: Guest

Access tokens

Kernel objects that contains the **security context** of a process|thread

- User SID $\approx$ Unix UID
- Group SIDs $\approx$ Unix GIDs
- Integrity level: Low, Medium, High, or System
 - used by Mandatory Integrity Control
- Privileges $\approx$ POSIX capabilities, i.e. the ability of doing system-level operations; e.g. shutdown, load-driver, take-ownership, ...
- ...

Types

- Primary Token: associated with a process
- Impersonation Token: associated with a thread
 - Can temporarily “act as” another user $\approx$ set-user-ID

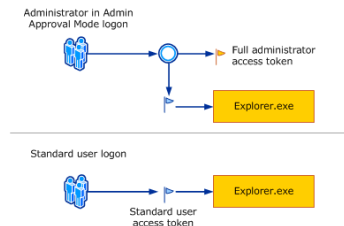
Logon

When you log in, your **security context** is encoded in an **access token**

- Before Windows Vista (2007) all users were administrators (!)
- Today, **administrators do not operate with admin rights by default**

When an administrator logs in, *two* access tokens are created

- One is the “real” admin token
- The other, used by default, has the rights of normal users

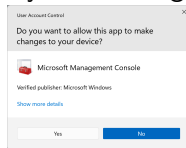


<https://docs.microsoft.com/en-us/windows/security/identity-protection/user-account-control/how-user-account-control-works>

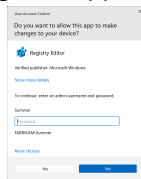
User Access Control

When a process needs admin rights, it requests elevation $\approx$ sudo

- If admin, with default UAC settings, a yes-no dialog-box pops up



- Otherwise, a username/password dialog box appears:



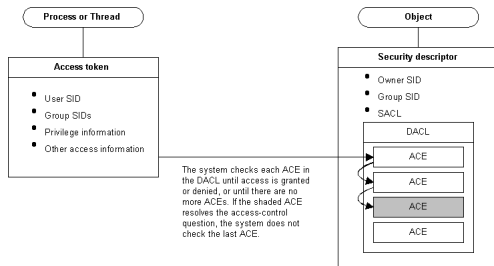
but, with default settings can be bypassed 😂

- <https://devblogs.microsoft.com/oldnewthing/20160816-00/?p=94105>
- <https://github.com/hfiref0x/UACME>

Tokens and Security Descriptors

Objects have security descriptors, specifying

- Owner/Group SID
- System ACL (SACL), indicating which operations should be logged
- Discretionary ACL (DACL)
 - Encoded as sequences of Access Control Entries (ACE)

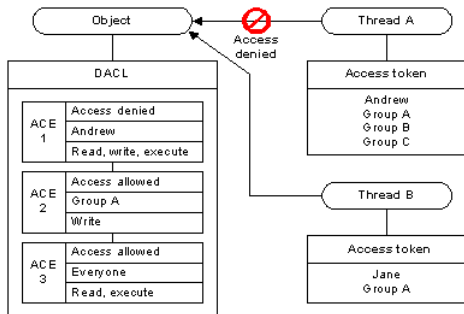


<https://docs.microsoft.com/en-us/windows/win32/secauthz/interaction-between-threads-and-securable-objects>

ACLs and ACEs

More fine-grained than the Unix one

- Each ACE allows/denies an operation for a SID
- Break down permissions into many more categories than just rwx
 - E.g., delete, change ownership, list folder, ...
- The system ensures Deny entries are the first ones



Universal Windows Platform applications

- AKA: metro, store, modern (apps) . . . *whatever*
 - e.g. the Calculator on Windows 11
- are akin to Android/mobile apps
 - have a **package-name**
 - **must declare capabilities** $\approx$ Android permissions
 - sandboxed inside an **AppContainer**
 - can be automatically suspended/resumed/terminated by the system

Mandatory Integrity Control

MIC allows Windows to differentiate processes run by the same user

- Integrity levels, identified by SIDs, are:
 - System system processes and services
 - High processes running elevated
 - Medium processes running with standard rights
 - Low AppContainer for UWP apps/Internet browsers
 - Untrusted Even more restricted sandboxes
- Objects without an integrity-level are considered Medium
- Files have no integrity-level by default

Protected Processes

Moreover, some processes are “special”

- Windows Vista introduced **protected processes** for DRM; they
 - must be specially signed (and can only load signed DLLs)
 - can only be opened with limited access, even by admins
- Windows 8.1 generalizes the idea into **PPL (Protected Process Light)** for 3rd-party anti-malware solutions and system services
 - Cannot be opened/terminated, even by admins
 - E.g., MsMpEng.exe (Microsoft Defender Antivirus)

<https://docs.microsoft.com/en-us/windows/win32/services/protecting-anti-malware-services->

Bring Your Own Vulnerable Driver (BYOVD)

How can you “fight” a PPL process, like an EDR?

- Drivers (.SYS files) run in kernel mode, the same PPL space
 - Driver: software that communicate with hardware devices by translating generic OS requests into device-specific instructions
- $\implies$ drivers can tamper with kernel memory/code
- Kernel Mode Code Signing: only signed drivers can be loaded
 - You need to get a production-signed by Microsoft
 - $\implies$ not easy to get a malicious driver signed by Microsoft
- Solution:
 - 1 Find a vulnerability in a driver
 - 2 Or use a driver with a known vulnerability that has not yet been banned by Microsoft
 - <https://www.loldrivers.io/>
 - 3 Exploit the vulnerability
 - 4 Enjoy kernel mode privileged actions

More details: “Unveiling BYOVD Threats” [MPO⁺26]

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 **Virtual Memory**
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Virtual Memory & Address Space

- **Virtual Memory**

- Provides isolation and protection between processes
- Gives each process the illusion of a large, continuous memory block
- Divided into atomic units: **pages**
- Permits to extend physical RAM using disk
 - Linux: swap partition
 - Windows: pagefile.sys file

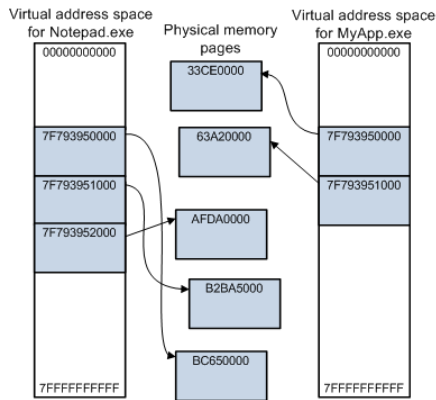
- **Virtual Memory Address Space**

- The range of addresses a process can use (logical|virtual addresses)
 - **VA** in many tools
- Illusion of much larger than physical RAM
 - E.g., 64-bit CPU $\Rightarrow$ up to 2^{64} addresses
- OS + Memory Management Unit (hardware, well-explained in [OB22])
 - Translates VAs $\rightarrow$ Physical addresses in RAM

Real-world details can be overwhelming; e.g., [Mar12] is an entire book on how VM works on Windows 7

Pagination

- Memory is divided into fixed-size blocks
 - Called pages (virtual) and frames (physical)
- Page size = size of each block
 - Same for pages and frames, so they can map 1:1
 - Modern OS use a default page size of 4 KB (4096 bytes)



Address spaces

Each process has its own isolated **address space**

When a program is executed, a new address space is built with:

- The **program** (code and data)
- Its required **libraries**
- **Runtime data**: stacks, heaps, env. variables, cmd-line arguments, ...
- The **kernel** 🤖
 - In the higher memory addresses, but only accessible in kernel mode
 - Modern systems add isolation mechanisms
 - Especially after CVE-2017-5754 <https://meltdownattack.com/>
 - Linux: Kernel Page-Table Isolation
 - Windows: Kernel Virtual Address Shadowing

User space processes' virtual address layout

The range of valid virtual addresses changes over time, as

- stacks grow/shrink
- threads are created/terminated
- dynamic libraries are (un)loaded
- heap memory is (de)allocated
- memory is (un)mapped

Dynamically Linked Libraries and Memory Sharing

- Code pages of executables and libraries are typically **read-only**
- Read-only pages can be **shared at the physical level** among processes
- Sharing occurs when processes:
 - run the same program
 - use the same dynamically linked libraries

However,

- Each process has its own **virtual address space**
- The same library may be mapped at **different virtual addresses**
- Pages that require **relocation** (code modification):
 - cannot be physically shared

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Types of Dynamic Memory APIs

Programmers have several APIs for dynamic memory allocation/protection

- Allocate memory at the page granularity (usually 4 KB)
 - You can allocate less than 4 KB, but the OS still allocate an entire page
- Allocate memory from heap
 - Windows: allocator (`RtlHeap`) lives in `ntdll.dll`, but heaps are also exposed as kernel objects, and so orchestrated by the OS runtime
 - Linux: heaps are purely implemented in the `libc.so` library
- Map a file or pagefile-backed region into memory; useful for
 - Memory-mapped files
 - Shared memory region
- Change protection flags on a memory region already allocated
 - Standard (`rxw`) on Linux
 - Windows some more: `GUARD`, `NOCACHE`, and `WRITECOMBINE`

Dynamic memory allocation (Windows)

VirtualAlloc → VirtualFree

```
#include <windows.h>
#include <stdio.h>

int main(void) {
    SYSTEM_INFO si; GetSystemInfo(&si);
    SIZE_T sz = si.dwPageSize;

    void *p = VirtualAlloc(NULL, sz,
                           MEM_COMMIT | MEM_RESERVE,
                           PAGE_READWRITE); // permissions
    if (!p) { fprintf(stderr, "VirtualAlloc failed\n"); return -1; }

    ((char*)p)[0] = 42; // use p...

    VirtualFree(p, 0, MEM_RELEASE);
    return 0;
}
```

Dynamic memory allocation (Linux)

mmap → munmap

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>

int main(void) {
    long page_size = sysconf(_SC_PAGESIZE);
    if (page_size == -1) { perror("sysconf"); return -1; }

    void *p = mmap(NULL, page_size,
                   PROT_READ | PROT_WRITE, // permissions
                   MAP_PRIVATE | MAP_ANONYMOUS, // anonymous memory
                   -1, 0);
    if (p == MAP_FAILED) { perror("mmap"); return -1; }

    ((char*)p)[0] = 42; // use the memory...
    if (munmap(p, page_size) == -1) { perror("munmap"); return -1; }
    return 0;
}
```

Dynamic memory allocation on the heap (Windows)

GetProcessHeap → HeapAlloc → HeapFree

```
#include <windows.h>
#include <stdio.h>

int main(void) {
    HANDLE hHeap = GetProcessHeap(); // handle to the heap kernel object
    if (!hHeap) { fprintf(stderr, "GetProcessHeap failed\n"); return -1;}

    void *p = HeapAlloc(hHeap, HEAP_ZERO_MEMORY, 1024);
    if (!p) { fprintf(stderr, "HeapAlloc failed\n"); return -1; }

    ((char*)p)[0] = 42; // use the memory...

    if (!HeapFree(hHeap, 0, p)) {
        fprintf(stderr, "HeapFree failed\n"); return -1; }
    return 0;
}
```

Dynamic memory allocation on the heap (Linux/Windows)

malloc → free

```
#include <stdlib.h>
#include <stdio.h>

int main(void) {
    size_t sz = 1024; // 1 KB
    void *p = malloc(sz);
    // Linux: libc manages the heap
    // Windows: under the hood uses HeapAlloc
    if (!p) { perror("malloc"); return -1; }

    ((char*)p)[0] = 42; // use the memory...

    free(p);
    return 0;
}
```


Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Address Space Layout Randomization (ASLR)

ASLR is technique supported by the OS

*In order to prevent an attacker from reliably jumping to, for example, a particular exploited function in memory, ASLR **randomly arranges the address space** positions of key data areas of a process, including the base of the executable and the positions of the stack, heap and libraries*

`https://en.wikipedia.org/wiki/Address\_space\_layout\_randomization`

Position-Independent Executable (PIE)

While the OS randomizes the location of stack, heap, and shared libraries

- PIE executables are built so that all code uses **relative addressing**
 - The loader can place them at a **random base address**
 - This enables **full ASLR**, including the main binary
 - The **entire address space** is randomized
- The compiler is in charge of generate PIE executables
 - Linux/GCC: `-fPIE -pie`
 - Windows/MSVC: `/DYNAMICBASE`
- Non-PIE executables are linked at a **fixed base address**

ASLR in Windows

Similar to Linux's ASLR, but more limited

- program base addresses are randomized per process
 - if program header contains the flag `IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE`
 - in theory... in practice, you may get the same address 🤔
 - uses “real” 64-bit addresses only with `IMAGE_DLL_CHARACTERISTICS_HI_ENTROPY_VA`
- (kernel and) **DLLs are randomized only once per boot**
 - DLLs remain shared in physical memory and relocated only once
- stacks and heaps are randomized

For more details, see

- [YSI17]
- <https://msrc-blog.microsoft.com/2013/12/11/software-defense-mitigating-common-exploitation-techniques/>
- <https://www.fireeye.com/blog/threat-research/2020/03/six-facts-about-address-space-layout-randomization-on-windows.html>

Disabling ASLR

Good practice for Reverse Engineering (RE) $\implies$
many addresses|offsets will not change between different executions

- For single programs you can change their flags
 - E.g., Windows: `IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE`
- But it is good practice not to modify the software under analysis
- For RE, disable it permanently at OS level (requires root/administrator)
- Linux:
 - `echo "kernel.randomize_va_space = 0" | sudo tee /etc/sysctl.d/01-disable-aslr.conf && sudo sysctl -system`
- Windows:
 - `Set-ProcessMitigation -System -Disable ForceRelocateImages`
 - `Set-ProcessMitigation -System -Disable BottomUp`

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Exceptional situations

What happens when...

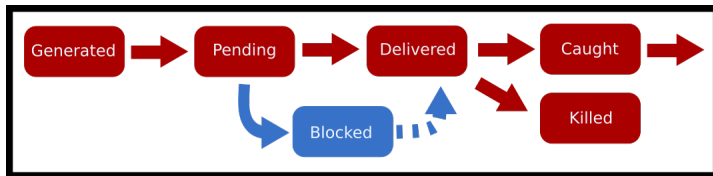
- a process divides by zero?
- a user press `Ctrl+C` on the terminal?
- a process writes over “read only” data?
- ...

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

Signals

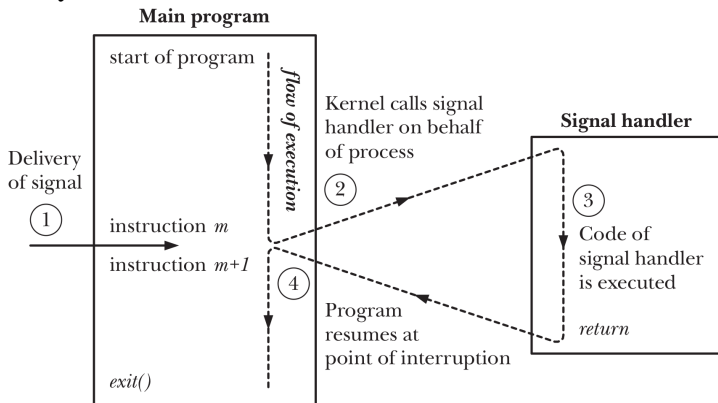
- **Asynchronous notifications** that “something” has happened
- **Identified by a number**, defined in `signal.h`, see `signal(7)`
- Can be sent by
 - The kernel
 - Another process, using `kill(2)`
- Signals are not queued: one signal may correspond to multiple events



From <https://github.com/illinois-cs241/coursebook>

Signal actions and handlers

- Each signal has a **default action**:
 - Ignore, terminate (and core-dump), stop/continue
- Some signals can be **handled** by registering a **signal handler** with `sigaction(2)` — see also `signal-safety(7)`



Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

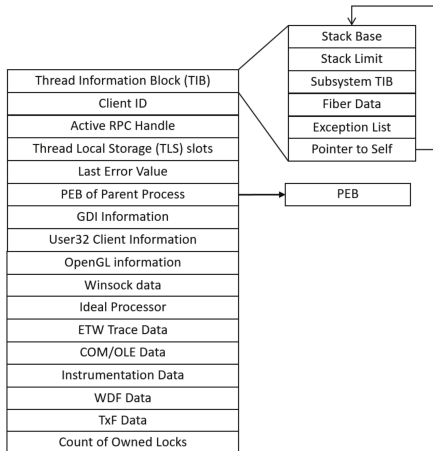
Background (Windows): TEB (Thread Environment Block)

Per-thread data structure in user memory

- On x86, there is a dedicated register (**FS** on x32 – **GS** on x64)
 - It does not contain a memory address
 - It is an index into a table of segment descriptors
 - and other info ...
- Stores thread-local data:
 - TIB (Thread Information Block)
 - **ExceptionList** (x32): head of SEH linked list (next slide)
 - **StackBase** / **StackLimit**: bounds of the thread stack
 - ...
 - **PEB (Process Environment Block)**: process-wide runtime information
 - **Thread Local Storage (TLS) array**
 - Mechanism to give each thread its own copy of a variable
 - ...

Exception (Handler) List

These registrations are codified by adding an **Exception Registration Record** in the LIFO (badly named) “Exception List”, pointed by the TIB

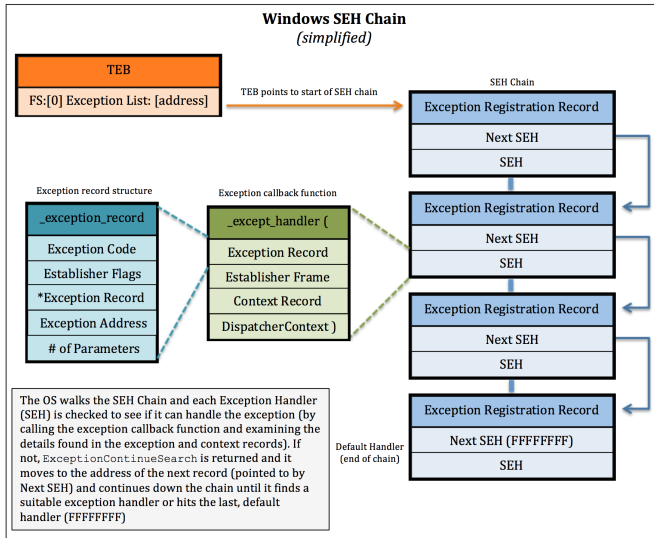


Structured Exception Handling (SEH)

Classic Windows mechanism for handling both HW and SW exceptions

- Purpose: localized exception handling for **specific code regions**
- Compiler support via `try / except / finally`
- On x86
 - 32: per-thread linked list of handlers
 - stored in the TIB→ExceptionList
 - 64: table-based
 - unwind metadata in sections `.pdata` or `.xdata`
- Can catch synchronous and asynchronous exceptions
 - Synchronous: happen as a direct result of executing an instruction
 - e.g., access violation, divide by zero
 - Asynchronous: external event
 - e.g., debugger breakpoints, `RaiseException`

Registration records (x86)



Vectored Exception Handling (VEH)

Introduced in Windows XP (2001) as a global exception mechanism

- **Process-wide**: not tied to a specific block of code
- Handlers registered/unregistered via APIs:
 - `AddVectoredExceptionHandler`
 - `RemoveVectoredExceptionHandler`
- Invoked **before SEH** during exception dispatch
- Purpose:
 - Logging, debugging, sandboxing, instrumentation
 - Similar conceptually to Unix signals

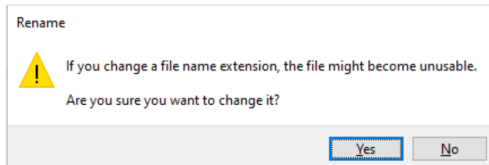
Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

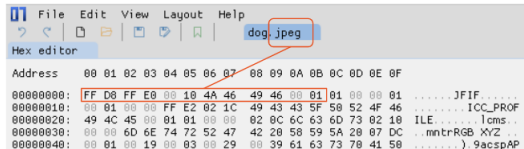
How information is *structured* for storage in a computer file.

Given a file, how can one determine its format?

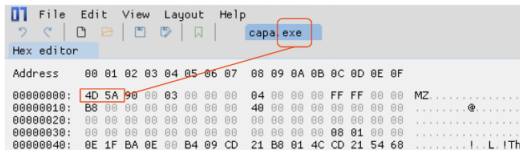
- **Magic Number|Bytes:** byte sequence in a certain position,
 - Often at the very beginning
 - Unique for each file type $\implies$ AKA **File Signature**
 - E.g., 4D 5A for Portable Executable (.exe)
 - E.g., 25 50 44 46 2D for Portable Document Format (.pdf)
- **File Extension:** a suffix to the name of a computer file
 - E.g., .pdf, .mp3, .exe



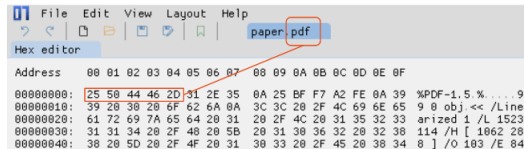
Examples of Magic Bytes



A screenshot of a hex editor window titled "dog.jpeg". The menu bar includes File, Edit, View, Layout, and Help. The toolbar shows various icons, and the file name "dog.jpeg" is highlighted. Below the toolbar, the text "Hex editor" is displayed. The main area shows a hex dump with addresses from 00000000 to 00000040. The hex values are: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F. The corresponding ASCII values are: FF D8 FF E0 10 4A 46 49 46 00 01 01 00 00 01 00 00 01 00 01 00 00 FF E2 02 1C 49 43 43 5F 50 52 4F 46 49 4C 45 00 01 01 00 00 02 0C 6C 63 6D 73 02 10 00 00 6D 6E 74 72 52 47 42 20 58 59 5A 20 07 DC 00 01 00 19 00 03 00 29 00 39 61 63 73 70 41 50. The ASCII values are:JFIF.....ICC_PROFILE.....lcms.. ..mnrRGB XYZ..).9acspAP



A screenshot of a hex editor window titled "capa.exe". The menu bar includes File, Edit, View, Layout, and Help. The toolbar shows various icons, and the file name "capa.exe" is highlighted. Below the toolbar, the text "Hex editor" is displayed. The main area shows a hex dump with addresses from 00000000 to 00000040. The hex values are: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F. The corresponding ASCII values are: 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 MZ..... B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68!..L !Th



A screenshot of a hex editor window titled "paper.pdf". The menu bar includes File, Edit, View, Layout, and Help. The toolbar shows various icons, and the file name "paper.pdf" is highlighted. Below the toolbar, the text "Hex editor" is displayed. The main area shows a hex dump with addresses from 00000000 to 00000040. The hex values are: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F. The corresponding ASCII values are: 25 50 44 46 2D 31 2E 35 0A 25 BF F7 A2 FE 0A 39 %PDF-1.5%. 39 20 30 20 6F 62 6A 0A 3C 3C 20 2F 4C 69 6E 65 9 0 obj << /Line 61 72 69 7A 65 64 20 31 20 2F 4C 20 31 35 32 33 arized 1 /L 1523 00000030: 31 31 34 20 2F 48 20 5B 20 31 30 36 32 20 32 38 114 /H [1062 28 00000040: 38 20 5D 20 2F 4F 20 31 30 33 20 2F 45 20 38 34 8] /O 103 /E 84

Example: dog.jpeg



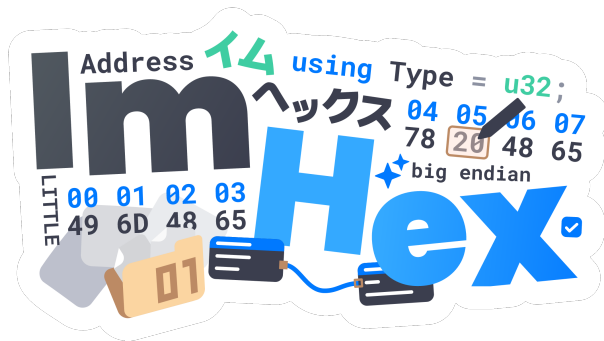
Example: structure of dog.jpeg

Hex editor																	
Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00000000:	FF	D8	FF	E0	00	10	4A	46	49	46	00	01	01	01	2C		...JFIF...
00000010:	01	2C	00	00	FF	E1	00	AE	45	78	69	66	00	00	49	49	...Exif.II
00000020:	2A	00	08	00	00	00	07	00	12	01	03	00	01	00	00	00	*.....b.....
00000030:	01	00	00	00	1A	01	05	00	01	00	00	00	62	00	00	00	j.....
00000040:	18	01	05	00	01	00	00	00	6A	00	00	00	28	01	03	00	1.....
00000050:	01	00	00	00	02	00	00	00	13	01	02	00	00	00	00	00	r..2.....
00000060:	72	00	00	00	32	01	02	00	14	00	00	00	80	00	00	00	i.....
00000070:	69	87	04	00	01	00	00	00	94	00	00	00	00	00	00	00	2.....
00000080:	2C	01	00	00	01	00	00	00	2C	01	00	00	01	00	00	00	GIMP 2.10.36.20
00000090:	47	49	40	50	20	32	2E	31	30	2E	33	36	00	00	32	30	25:11:09 12:35:0
000000A0:	32	35	3A	31	31	3A	30	39	20	31	32	3A	33	35	3A	30	...http://n
000000B0:	32	00	01	00	01	A0	03	00	01	00	00	00	01	00	00	00	s.adobe.com/xap/
000000C0:	00	00	00	00	FF	E1	0C	CF	68	74	74	70	3A	2F	2F	6E	1.0/ <?xpacket b
000000D0:	73	2E	61	64	6F	62	65	2E	63	6F	6D	2F	78	61	70	2F	egin="..." id="W
000000E0:	31	2E	30	2F	00	3C	3F	78	70	61	63	68	65	74	20	62	5M0MpCehiHzreSzN
000000F0:	65	67	69	6E	3D	22	EF	BB	8F	22	20	69	64	3D	22	57	Iczkc9d"?> <x:xm
00000100:	35	4D	30	4D	70	43	65	68	69	48	7A	72	65	53	7A	4E	pmeta xmlns:x="a
00000110:	54	63	7A	6B	63	39	64	22	3F	3E	20	3C	78	3A	78	6D	dobe:ns:meta/" x
00000120:	70	6D	65	74	61	20	78	6D	6C	6E	73	7A	78	3D	22	61	:xmptk="XMP Core
00000130:	64	6F	62	65	3A	6E	73	3A	6D	65	74	61	2F	22	20	78	
00000140:	3A	78	6D	70	74	6B	3D	22	58	4D	50	20	43	6F	72	65	

Pattern Data				
Name	Co...	Start	End	Size
▼ segments		0x00000000	0x000AA92A	698667 bytes
★ ▼ [0]		0x00000000	0x00000001	2 bytes
★ ▼ [1]		0x00000002	0x00000013	18 bytes
★ ▼ [2]		0x00000014	0x000000C3	176 bytes
★ magic		0x00000014	0x00000014	1 byte
★ marker		0x00000015	0x00000015	1 byte
★ length		0x00000016	0x00000017	2 bytes
★ data		0x00000018	0x000000C3	172 bytes
★ ▼ [3]		0x000000C4	0x00000094	3281 bytes
★ ▼ [4]		0x00000095	0x00001046	690 bytes
★ ▼ [5]		0x00001047	0x0000108B	69 bytes
★ ▼ [6]		0x0000108C	0x000010D0	69 bytes
★ ▼ [7]		0x000010D1	0x000010E3	19 bytes
★ ▼ [8]		0x000010E4	0x00001103	32 bytes
★ ▼ [9]		0x00001104	0x00001122	31 bytes
★ ▼ [10]		0x00001123	0x000AA928	694278 bytes

Structure of a File Format

A file structure is the organized layout of data and metadata within a file that defines how its contents are stored and interpreted by software



It is in another slide deck... [👉 \[link \]](#)

Magic Database File

Magic DB is the most used database of file signatures

- `libmagic` is a library that identifies file types
- Unix `file` program uses `libmagic`
- In Python the `python-magic` package wraps `libmagic`

Problem: file signatures, although unique, do not necessarily appear at the beginning of the files (e.g., ZIP) $\implies$

Although a file has only one extension, it can be of “multiple types”

Multi-Format Container and Polyglot Files

Multi-Format Container *reuses or extends* another format (often ZIP) to package structured data with specific conventions. E.g.:

- Android Package APK are Java ARchives JAR that in turn are ZIP
- Office Open XML (OOXML) by Microsoft
 - E.g., .docx, .pptx, and .xlsx

A **polyglot** file is crafted to be valid under two or more *independent* file formats simultaneously. E.g.:

- GIFAR is both a GIF and a RAR file
- Phar-JPEG file are used to carry out PHP object for injection attacks

Unlike containers, which are legitimate and standardized, polyglots often appear in malicious contexts to bypass filters, evade detection systems, or steganography

How to determine the file type?

- Unix file --keep-going
 - Do not stop at the first match, keep going
- **Magika** by Google – ML-based, fine grained
 - <https://github.com/google/magika>
 - pip install magika
- 🍰 **PolyFile** by Trail of Bits 🍰
 - <https://github.com/trailofbits/polyfile>
 - pip install polyfile

```
# wget https://www.sultanik.com/files/ESultanikResume.pdf
# polyfile ESultanikResume.pdf
NES ROM image (iNES): 8x16k PRG, 2x8k CHR
PDF document, version 1.5, 3 pages
Malformed PDF
Zip archive data with 57041 byte comment
ZIP end of central directory record
```

Outline

- 1 Library and System calls
- 2 Processes and Threads
- 3 Authorization
 - Linux
 - Windows
- 4 Virtual Memory
 - Memory APIs
 - Address Space Layout Randomization (ASLR)
- 5 Dealing with “exceptional” situations
 - Linux Signals
 - Windows Exception Handling
- 6 File Formats
 - Windows Portable Executable

The Portable Executable (PE) file format

File format for native executable code on Windows OSs and UEFI

- Executables (.exe)
- Dynamically-linked library (.dll)
- Device Drivers (.sys)

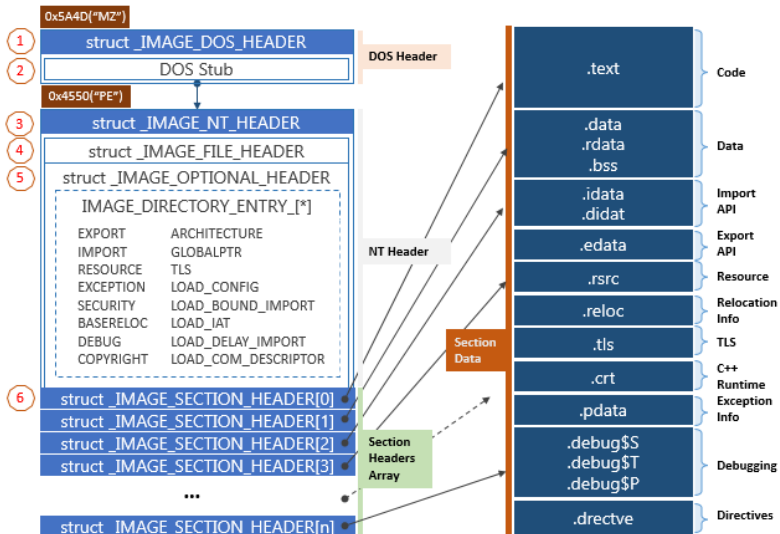
Adopted by Microsoft since the release of Windows 3.1 🤖

- Extension of DOS **MZ** and Common Object File Format (COFF)
 - **Mark Z**bikowski
- Can run on both MS-DOS (see DOS stub later) and Windows
- 32/64-bit format: **PE32** and **PE32+** (not PE64)
- Data structures can be found in **winnt.h** from Windows SDK

The **image** is the in-memory representation of a PE file after it has been loaded by the operating system

PE Format — Structure

Its design consists of a number of structured data types named *headers*



① DOS Header

```
typedef struct _IMAGE_DOS_HEADER {  
    WORD    e_magic;           // Magic number MZ = 4D 5A  
    // ...  
    LONG    e_lfanew;          // Offset|pointer to PE header  
} IMAGE_DOS_HEADER, *PIMAGE_DOS_HEADER;
```

- First header at the very beginning of the file
- Originally used in the MS-DOS operating system
- Still in use today for backward compatibility
- For Windows, only used for the offset at which the NT header starts

② DOS Stub

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00000000:	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ.....
00000010:	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00000020:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030:	00	00	00	00	00	00	00	00	00	00	00	00	E8	00	00	00	
00000040:	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	!..L.!Th
00000050:	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program canno
00000060:	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070:	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode....\$.

Still part of the DOS header, contains a 16-bit MS-DOS executable

- Usually prints “This program cannot be run in DOS mode” and exits
- It can be a full-fledged DOS version of the program
 - Option /STUB of Microsoft linker
 - Attaches an MS-DOS stub program to a Windows program

2-3 The “Rich” header

The Rich header is located after the DOS stub and before the PE header

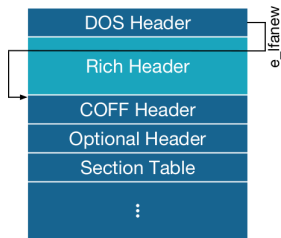


Fig. 1. High level view of the *PE32* format

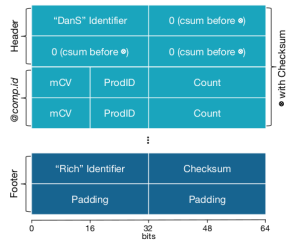


Fig. 2. Structure of the Rich Header

Undocumented header, inserted by MS linker with information about the build environment

- Can remain unchanged for long time in the development cycle
- $\implies$ Rich PE header hash for triaging [WKvP⁺17]
- Some protectors (e.g., Enigma and Themida) preserve the original [AGM⁺20]

③ NT header and ④ Image File header

```
typedef struct _IMAGE_NT_HEADERS { // (3)
    DWORD Signature; // "PE\x00\x00" = 50 45 00 00
    IMAGE_FILE_HEADER FileHeader; // -> (4)
    IMAGE_OPTIONAL_HEADER32 OptionalHeader; // -> (5)
} IMAGE_NT_HEADERS32, *PIMAGE_NT_HEADERS32;
```

```
typedef struct _IMAGE_FILE_HEADER { // (4)
    WORD Machine;
    ...
    WORD Characteristics;
} IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

- Machine = architecture, e.g.:
 - 0x014c → x86
 - 0x8664 → x86-64
- Characteristics = file type, e.g.:
 - 0x0002 → EXE
 - 0x2000 → DLL

⑤ Image “Optional” header

```
typedef struct _IMAGE_OPTIONAL_HEADER {  
    ...  
    DWORD ImageBase;           // Base address (unless ASLR)  
    DWORD AddressOfEntryPoint; // RVA of very first instruction (*)  
    DWORD SectionAlignment;    // Alignment of sections in memory  
    DWORD FileAlignment;       // Alignment of sections on disk  
    DWORD SizeOfImage;         // Total size in memory  
    WORD Subsystem;            // GUI/CLI/EFI/NATIVE ...  
    WORD DllCharacteristics;    // Bitmask with various flags (*)  
    IMAGE_DATA_DIRECTORY DataDirectory[16]; // (*)  
    ...  
} IMAGE_OPTIONAL_HEADER32, *PIMAGE_OPTIONAL_HEADER32;
```

“(*)” means “discussed in the next slides”

Relative Virtual Address (RVA): offset of a memory location from the ImageBase
Virtual Address (VA): ImageBase + RVA

AddressOfEntryPoint → CRT → *Main()

The entry point usually points to C Run-Time Library (CRT) code which, in turn, calls main/WinMain/DllMain

- **Console:** _mainCRTStartup → main()
- **Windows GUI:** WinMainCRTStartup → WinMain()
- **DLLs:** _DllMainCRTStartup → DllMain()

```
BOOL WINAPI DllMain(HINSTANCE hInstDll, DWORD reason, PVOID reserved)
```

Its arguments are the base address of the DLL (hInstDll) and a reason:

`DLL_PROCESS_ATTACH (1)` the DLL has been attached to a process

`DLL_PROCESS_DETACH (0)` the DLL has been unloaded

`DLL_THREAD_ATTACH (2)` a new thread has been created

`DLL_THREAD_DETACH (3)` a thread is exiting

Despite “Dll” prefix, it is present in both EXEs and DLLs.

Some security-related flags `IMAGE_DLLCHARACTERISTICS_`:

- `DYNAMIC_BASE`: enables ASLR
- `HIGH_ENTROPY`: supports high-entropy 64-bit ASLR
- `GUARD_CF`: supports Control Flow Guard (CFG)
- `NX_COMPAT`: compatibility with Data Execution Prevention (DEP)

DataDirectory[16]

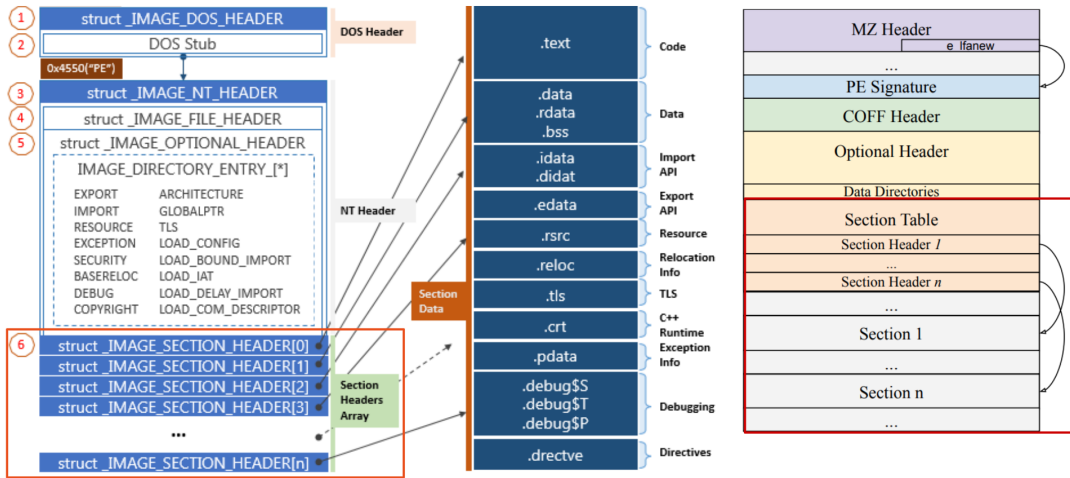
IMAGE_OPTIONAL_HEADER structure. Defines an array of 16 entries, i.e.:

```
typedef struct _IMAGE_DATA_DIRECTORY {  
    DWORD    VirtualAddress;  
    DWORD    Size;  
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;
```

Most relevant 'Tables' are:

- 0 **Export**: symbols exported (e.g., DLL functions)
- 1 **Import**: symbols imported from other DLLs
- 2 **Resource**: icons, bitmaps, raw bytes, etc.
- 4 **Certificate**: digital signatures (MS Authenticode)
- 5 **Base Relocation**: fix addresses when not loaded at its ImageBase
- 9 **Thread Local Storage (TLS)**: thread-local variables initialization; custom callback functions can be registered to execute 🤖
- 13 **Import Address Table (IAT)**: runtime addresses for imported functions (populated during loading)

⑥ Section Table



Section headers

- PointerTo* *rightarrow* on disk
- Virtual* $\rightarrow$ in memory

```
#define IMAGE_SCN_MEM_EXECUTE    0x20000000    // Section is executable
#define IMAGE_SCN_MEM_READ      0x40000000    // Section is readable
#define IMAGE_SCN_MEM_WRITE     0x80000000    // Section is writeable
...
typedef struct _IMAGE_SECTION_HEADER {
    BYTE    Name[8];                // String (max 8 chars)
    DWORD   VirtualSize;
    DWORD   VirtualAddress;        // RVA
    DWORD   SizeOfRawData;
    DWORD   PointerToRawData;
    DWORD   PointerToRelocations; // file offset to the relocation table
    WORD    NumberOfRelocations;
    ...
    DWORD   Characteristics; // bitmask of IMAGE_SCN_...
} IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;
```

Data Execution Prevention (DEP)

DEP is the MS Windows implementation of executable-space protection

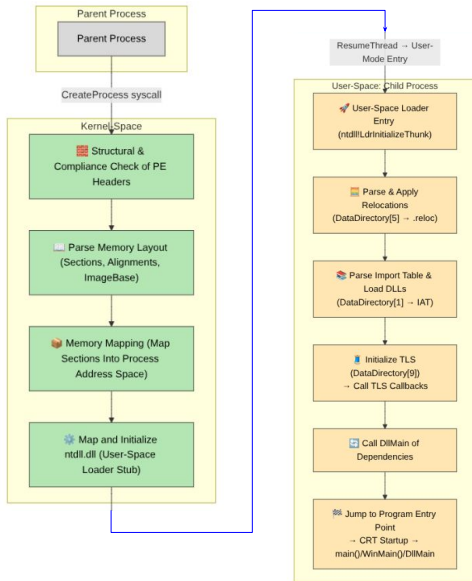
- Marks memory regions as non-executable
- An attempt to execute code in these regions will cause an exception
- Implemented with hardware features (NoeXecute bit) or software

Windows supports 4 system-wide configurations, and by default:

- DEP is enabled for limited system binaries and programs that “opt-in”
- How to “opt-in”? `NX_COMPAT` in `DllCharacteristics`

When Data Execution Prevention is disabled, every section is RWX regardless of its original protection attributes defined in its `Characteristics` field

Windows Loading Process [NGFB21]



Windows Loading Process of a PE executable – Recap

A process creates another process; e.g.:

- Double-clicking on an icon (of an executable) from Explorer
- Running a command from PowerShell

The **kernel**

- Parses the PE and allocates memory accordingly
- Copies and initialize `ntdll.dll`
- Finally, pass control to the `ntdll.dll` code in user space

In **user space**, starting from `ntdll.dll!LdrInitializeThunk`

- If ASLR, applies relocations of memory addresses
 - Local function calls and variables
- Parses the import tables
 - Loads the needed DLLs
 - Fix entries in the Import Address Table (IAT) of external functions
- Parses the Thread Local Storage (TLS)
 - If there are callback functions, they are executed $\implies$ **entry point!**
- Runs initialization code of imported DLLs
- Jumps to the entry point

- [AGM⁺20] Hojjat Aghakhani, Fabio Gritti, Francesco Mecca, Martina Lindorfer, Stefano Ortolani, Davide Balzarotti, Giovanni Vigna, and Christopher Kruegel.
When malware is packin' heat; limits of machine learning classifiers based on static analysis features.
In *Network and Distributed System Security Symposium*. Internet Society, 2020.
- [Ker10] Michael Kerrisk.
The Linux programming interface: a Linux and UNIX system programming handbook.
No Starch Press, 2010.
- [Mar12] Enrico Martignetti.
What Makes It Page?: The Windows 7 (x64) Virtual Memory Manager.
CreateSpace Independent Publishing Platform, 2012.

References II

- [MPO⁺26] Andrea Monzani, Antonio Parata, Andrea Oliveri, Simone Aonzo, Davide Balzarotti, Andrea Lanzi, et al.
Unveiling byovd threats: Malware's use and abuse of kernel drivers.
In Network and Distributed System Security (NDSS) Symposium 2026. NDSS, 2026.
- [NGFB21] Dario Nisi, Mariano Graziano, Yanick Fratantonio, and Davide Balzarotti.
Lost in the loader: The many faces of the windows pe file format.
In 24th International Symposium on Research in Attacks, Intrusions and Defenses, pages 177–192, 2021.
- [OB22] Andrea Oliveri and Davide Balzarotti.
In the land of mmus: multiarchitecture os-agnostic virtual memory forensics.
ACM Transactions on Privacy and Security, 25(4):1–32, 2022.

References III

- [TDSW08] Dan Tsafir, Dilma Da Silva, and David Wagner.
The murky issue of changing process identity: revising “setuid demystified”.
USENIX Login, 33(3):55–66, 2008.
- [WKvP⁺17] George D Webster, Bojan Kolosnjaji, Christian von Pentz, Julian Kirsch, Zachary D Hanif, Apostolis Zarras, and Claudia Eckert.
Finding the needle: A study of the pe32 rich header and respective malware triage.
In *International conference on detection of intrusions and malware, and vulnerability assessment*, pages 119–138. Springer, 2017.
- [YSI17] Pavel Yosifovich, David A Solomon, and Alex Ionescu.
Windows Internals, Part 1: System architecture, processes, threads, memory management, and more.
Microsoft Press, 2017.